

06-checkout信用卡智能扣款重试

目录

一. 需求背景	-----
二. 需求价值	-----
三. 需求描述	-----
• 3.1 核心目标	-----
• 3.2 EF实验逻辑	-----
• 3.3 EF实验观测指标	-----
四、注意：特殊场景	-----
• 4.1 重复扣款	-----
• 4.2取消checkout重试场景	-----

一. 需求背景

- **问题现状：**当前续费扣款重试为公司主动发起，基于固定时间间隔的重试策略（2+2+2+4+8+8），未充分结合发卡行行为、用户历史支付习惯、实时网络状态等数据进行智能决策。

二. 需求价值

- 提升续费扣款成功率

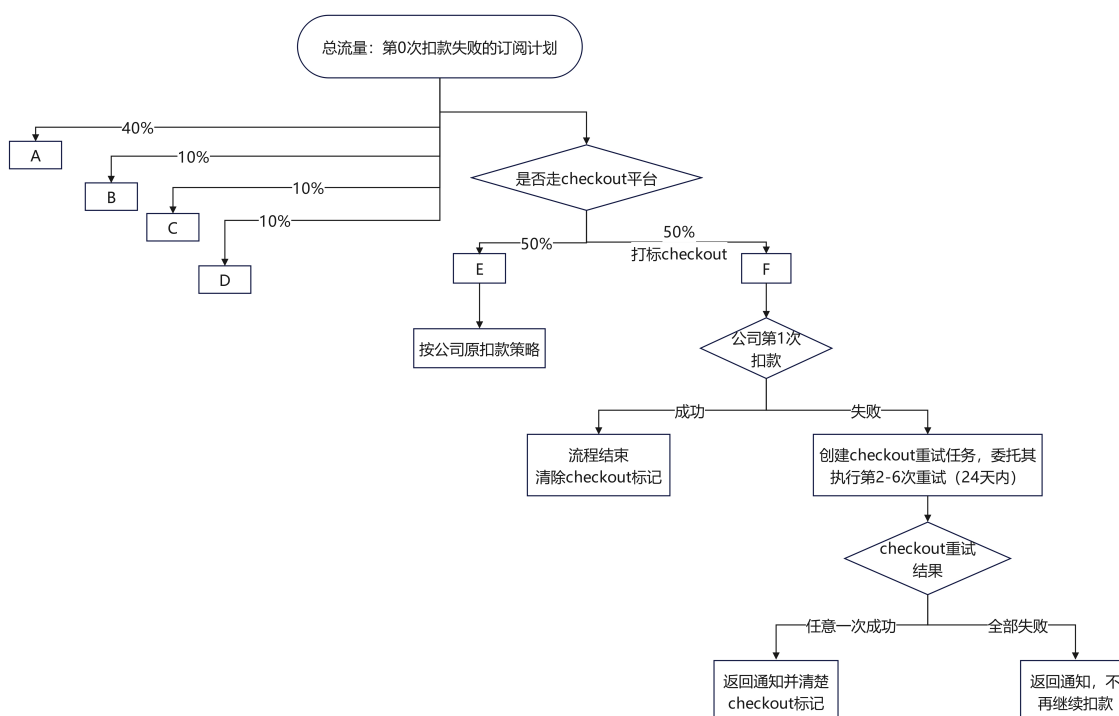
三. 需求描述

3.1 核心目标

在原有的AB实验中添加EF同时进行实验，但实验结果EF独立对比，找到续费成功率最大化的策略。

算法	核心逻辑
A(现有)	固定间隔2/2/2/4/8/8天，不考虑日期质量
B(智能)	- 优先选择黄金日期，避开黑名单日期，允许跳跃式重试 - 完全数据驱动
C(混合)	前两次保持2天固定间隔(稳定基础)，后续才启用智能选择
D(加权)	给每个日期打分(考虑成功率、距离、场景价值)，选择分数最高的日期
E (对照)	按公司原扣款策略固定间隔2/2/2/4/8/8天
F (实验)	第0、1次公司自扣，后续2-6次checkout智能扣款

3.2 EF实验逻辑



3.3 EF实验观测指标

- 采用EF实验验证不同时间段扣款对扣款成功率影响，E实验维持不变，F实验在最佳时间段扣款
- 定时任务提前1天分流时打标

实验	订阅计划数	成功计划数	总扣款次数	成功扣款次数	平均成功天数
E实验					
F实验					

四、注意：特殊场景

4.1 重复扣款

若在扣款时发生给用户重复扣款的场景，要为用户申请退款，但退款时不能取消该订阅计划

4.1.1前置多渠道扣款避免系统自动扣两次款

4.1.2重复扣款后定时任务自动退款

4.1.3自动退款时不取消订阅计划

4.2取消checkout重试场景

1. 用户主动操作类

场景	处理方式
(1) 用户在用户中心取消订阅	取消重试
(2) 用户提前完成付款	支付成功后自动终止重试；若已有重试安排，
(3) 用户更换信用卡	原卡支付失败且新卡已绑定，应取消旧卡重试

2. 异常场景

场景	说明	处理方式
差价付费升级	原订阅被替换为新订阅	取消原订阅的重试
赠送付费升级	免费升级至高阶服务	取消原订阅重试
订阅挽回活动	用户被挽留并续订	若原扣款未成功，取消重试
升级追加可变单位值	订阅内容变更导致原计划失效	取消原重试
平级切换	计划变更但周期不变	视为原订阅终止，取消重试
提前续费运营	系统提前触发续费	新支付成功后，取消旧失败
替换续费凭证	如发票、合同编号变更导致原扣款作废	取消原重试

七、完整流程图

quickPay 扣款流程



└ 有可用多渠道token?

└ 多渠道token扣款成功 → 直接返回成功

└ 多渠道token扣款失败 → 标记skip_multi_token_retry, 继续步骤2

└ 条件不满足 → 跳过, 继续步骤2

2. Checkout::quickPay() → CheckoutBase::_quickPay()

3. applyDunningIfEligible() 判断:

└ is_from_quickpay_retry? → 多渠道重试场景直接跳过

└ 配置开关ENABLED是否打开?

└ retry_num == TRIGGER_RETRY (当前配置=1)?

└ pay_method_id in PAY_METHOD_IDS (当前配置=[1,5])?

└ AB实验分流 (AB_TEST_ENABLED时复用续费金额微调实验分组)

4. AB实验开启时检查分组:

└ 查询 RENEWAL_DATE_ADJUSTMENT_TEST_CORN 标记

└ flag != F → 不启用dunning, 正常扣款

└ flag == F (实验组) → 进入步骤5

5. format(QUICK_PAY) 构建请求体

└ dunning_enabled=true → 带上 retry.dunning 参数

(end_after_days=24, max_attempts=5)

└─ dunning_enabled=false → 正常请求

6. callJsonPost() 请求Checkout API

7. 处理响应:

└─ 成功(code==0):

└─ 更新transaction状态为SUBMIT_SUCCESS

└─ 失败(code!=0):

└─ 存储响应id到pay_order.platform_no (用于后续取消催款)

└─ 带dunning且响应包含retry → updateDunningRemark()写入remark

└─ 多渠道重试(skip_multi_token_retry时跳过)

8. PaymentService层统一处理:

└─ 最终扣款成功(code==0) → clearDunning()

└─ 先cancelDunningOnCheckout() → 成功后clearDunningRemark()

└─ 最终扣款失败 → 保留dunning标识, 等待Checkout后续自动重试

取消智能催款的触发场景



复制代码

取消智能催款的触发场景

A. 续费扣款成功（含多渠道重试成功）

→ `cancelDunningOnCheckout()` 成功后 `clearDunningRemark()`

（`cancelDunningOnCheckout` 内置最多2次重试，403视为已取消）

B. `updatePlan/updateRenewal` 更新续费计划时：

└ `renewal_date` 变更且超过当前时间28天

└ `renewal_status` 变更为 `-1` 或 `-2`

└ 关键字段变化：`platform_renewal_ref/pay_method_id/`

`pay_platform/renewal_item_key/pay_id/amount_per_cycle/currency`

→ `cancelDunningOnCheckout()` 成功后 `clearDunningRemark()`

C. 取消续费计划 `cancelPlan`

→ `cancelDunningOnCheckout()` 成功后 `clearDunningRemark()`

D. `Checkout` IPN回调通知催款成功

→ 仅清除`remark`标识（不需要`void`）

八、涉及文件改动清单

序号	文件路径	改动类型	改动说明
1	config/checkout.php	修改	新增DUNNING配置项（TRIGGER_RETRY=1, MAX_ATTEMPTS=5, AB_TEST_ENABLED等）
2	app/Wondershare/Logic/Common/Components/CheckoutBase.php	修改	format() QUICK_PAY分支增加retry.dunning:platform_no存储及remark更新；新增 applyDunningFromQuickpayRetry排除和AB实验分流）、remark()（清空remark为空字符串）、cancelDunning()方法
3	app/Rpc/Service/v2/PaymentService.php	修改	quickPay() 中新增 tryDunningMultiTokenFirst() 逻辑；updateRenewal() 增加（公开RPC接口）、cancelCheckoutDunning() 方法
4	app/Wondershare/Logic/Common/Payment.php	修改	_updatePlan() 增加dunning取消判断；_cancelDunningAndCancelDunning()、cancelDunningViaCheckout()方法
5	Checkout IPN/Webhook处理文件	修改	催款成功回调时清除dunning标识
6	app/Wondershare/Logic/Common/TrafficCollecting.php	修改	新增 TRAFFIC_TYPE_CHECKOUT_DUNNING 常量；支持进行AB实验分组

九、测试要点

9.1 触发条件测试



9.2 Remark写入测试

- ☐ 扣款失败且响应含retry → remark应写入dunning标识（含 payment_id/max_attempts/next_attempt_on/ends_on）
- ☐ remark已有其他内容 → dunning标识应合并而非覆盖

9.3 清除标识+取消催款测试

- ☐ 多渠道重试成功 → Checkout取消催款请求发送成功后remark清空
- ☐ 原渠道扣款成功 → Checkout取消催款请求发送成功后remark清空
- ☐ Checkout取消催款API返回403 → 视为已取消，清除remark
- ☐ Checkout取消催款API失败（非403） → 重试最多2次；最终失败则不清除remark
- ☐ updatePlan: renewal_date变更且设为30天后 → 触发取消
- ☐ updatePlan: renewal_status变更为-1 → 触发取消
- ☐ updatePlan: renewal_status变更为-2 → 触发取消
- ☐ updatePlan: platform_renewal_ref变化 → 触发取消
- ☐ updatePlan: pay_method_id变化 → 触发取消
- ☐ updatePlan: pay_platform变化 → 触发取消
- ☐ updatePlan: renewal_item_key变化 → 触发取消
- ☐ updatePlan: pay_id变化 → 触发取消
- ☐ updatePlan: amount_per_cycle变化 → 触发取消
- ☐ updatePlan: currency变化 → 触发取消
- ☐ cancelPlan → 触发取消
- ☐ Checkout IPN回调催款成功 → 仅清除remark，不取消

9.4 AB实验测试

- ☐ AB_TEST_ENABLED=true时，查询
- ☐ 同一renewal_id复用已有的续费金额微调实验标记，不重新分流
- ☐ AB_TEST_ENABLED=false时 → 全量启用dunning（不做分组判断）

9.5 智能催款多渠道优先测试（tryDunningMultiTokenFirst）

- ☐ retry_num=1, pay_platform=checkout, pay_method_id in (1,5),
- ☐ 多渠道token扣款成功 → 直接返回成功，标记handleQuickPaySuccess
- ☐ 多渠道token扣款失败 → 标记skip_multi_token_retry，继续原渠道dunning流程
- ☐ 条件不满足（如flag!=B） → 跳过，走正常流程

9.6 边界场景测试

- ☐ remark为空字符串时的写入和读取

- ☐ remark为非JSON字符串时的兼容处理
 - ☐ Checkout取消催款请求最终失败时的容错处理（不应影响主流程，不清除remark）
 - ☐ dunning配置总开关关闭时不触发
 - ☐ clearDunningRemark 清除后remark设为空字符串（非保留其他键的JSON）
-